

5.9 Compose 2/2 — MVI под микроскопом: канонический MVICore, конкурентность, доставка News и Interview Prep — учебный конспект

TL;DR

- **Владение состоянием:** state machine живёт внутри `ActorReducerFeature` (MVICore 1.x, RxJava 2); `MviViewModel` — anti-corruption layer: зеркалит state в `StateFlow`, адаптирует news, освобождает feature в `onCleared`. Из канонической модели отброшены `WishAction`, `Bootstrapper`, `PostProcessor`, `Binder`, middleware и time-travel — вместе с их гарантиями и инструментами; их функции легли на UI и дисциплину команды.
- **Конкурентность:** сериализуется только применение `EFFECT`-ов на feature-потоке; сами `Actor`-ы исполняются параллельно (отдельная Rx-подписка на каждый action, без `concatMap`/`switchMap`) → out-of-order и stale results. Рабочая защита — `requestId` в `STATE`/`EFFECT` + guard в `Reducer`.
- **News:** связка `MutableSharedFlow(0, 0)` + `LinkedList` + `subscriptionCount` — best-effort-доставка: гонка check-then-emit, потеря элемента при отмене во время `dispatchQueue`, неограниченная память, broadcast всем коллекторам, replay устаревших событий после возврата на экран, испарение при process death. Для одного consumer'а честнее `Channel.receiveAsFlow()`, для навигации — durable state + acknowledgement.
- **Интервью:** наготове 30-секундный питч потока данных, шесть слабых мест с фиксами и три стратегии эволюции (укрепить контракты → изолировать Rx → coroutine-native UDF). Всё — в финальном блоке с вопросами, модельными ответами и ловушками.

Как читать этот конспект

Это вторая половина конспекта 5.8 (Compose 1/2) и четвёртый конспект серии. Первая часть *заявила* MVI-разбор в TL;DR и Key Findings (пункты 5–6: «четыре болевые точки MVVM», «anti-corruption layer + never throw + never-drop-очередь»), но детали остались за кадром — здесь мы их закрываем целиком. Предполагается знакомство с частью 1/2 (фазы рендеринга, два вида «пропусков», ниша XML-2026) и с предыдущими конспектами (стабильность/strong skipping, side-effect API, перф-плейбук, база Actor/Reducer/NewsPublisher). Формат прежний: для трудных мест сначала бытовая аналогия, затем точная формулировка. Код — Kotlin; стек проекта: MVICore-wrapper (`MviViewModel`), Dagger 2 без Hilt, Timber, JUnit5 + MockK, Fragment + ComposeView + кастомный FragmentAttacher.

Key Findings

1. **Владелец state — MVICore Feature** (`BehaviorSubject` внутри), а не `MutableStateFlow`: последний — асинхронное зеркало через адаптер. Это меняет подход к тестам (`advanceUntilIdle`), а не мгновенный (`state.value`), дебагу и мониторингу мостов.
 2. **Ваш wrapper** $\approx$ `ActorReducerFeature`: `Wish  $\equiv$  Action` (`identity`), `Bootstrapper`/`PostProcessor`/`Binder`/middleware отброшены. Стартовую загрузку инициирует UI (`ScreenOpened`), поэтому Actor обязан быть идемпотентным.
 3. **«Однопоточный feature» сериализует редукцию, но не Actor'ов.** Без `requestId`-guard'a возможны stale results и out-of-order-эффекты — это не гипотеза, а прямое следствие устройства `BaseFeature`.
 4. `ActorAdapter` (`asObservable(Dispatchers.Main.immediate)`) даёт structured cancellation и main-эмиссии «бесплатно», но намертво блокирует `FeatureScheduler`: даже прокинутый фоновый scheduler сломает thread-check. Фольклорное «переключитесь в конце на UI-поток» — упрощение реальной семантики.
 5. **Контракт «must never throw» относится к ожидаемым ошибкам.** `CancellationException` обязан пролетать (иначе ломается structured cancellation), программные дефекты должны быть fail-fast и наблюдаемыми, иначе — silent corruption.
 6. `catch { Timber.e(th) }` на мостах превращает первую же ошибку в «полуживую» ViewModel: feature жива, actions принимаются, но UI навсегда заморожен на последнем состоянии.
 7. **News-механизм — best-effort, а не exactly-once:** гонка check-then-emit, `poll()` до завершения `emit`, broadcast нескольким коллекторам, replay устаревшей навигации, unbounded-память. Очередь честно решает одну задачу — окно без подписчика при configuration change.
 8. **Process death сбрасывает всё в initialState.** Восстанавливайте recovery input через `SavedStateHandle`; записывать в него нужно из подписки во ViewModel, а не из pure `Reducer`.
 9. `WidgetMviViewModel` прячет unchecked cast — типобезопасность возвращается generic-параметром интерфейса, а `onCommand` (второй вход в state machine) обязан мапиться в `Action`, а не мутировать состояние напрямую.
-

Details

Тема 3. Почему единый STATE ложится на Compose — и где именно болит MVVM с N StateFlow

Закрываем пункт 5 из Key Findings части 1/2 — там он был заявлен без деталей.

Аналогия. Compose — это принтер, который умеет печатать только целую страницу по макету. MVVM с пятью `StateFlow` — это пять человек, которые одновременно диктуют принтеру правки в разные абзацы. MVI отдаёт принтеру ровно то, что он хочет: один готовый макет страницы на каждый момент времени.

Точная формулировка. Ментальная модель Compose: `UI = f(state)` — функция от снимка. Единый неизменяемый `STATE` из MVI и есть этот снимок: любое изменение проходит через один `Reducer`, поэтому в composition никогда не попадает «наполовину обновлённая» комбинация полей. Четыре точки, где MVVM с N независимыми потоками напрягается, а MVI закрывает конструктивно:

1. **Гонки без координатора.** Пять `MutableStateFlow` обновляются из разных корутин; UI может отрисовать кадр между обновлением `isLoading = false` и `items = newList`. В MVI переходы атомарны: один `EFFECT` → один новый `STATE`.
2. **Нечитабельные `combine()`.** Чтобы собрать экран из N потоков, пишут `combine(a, b, c, d, e) { ... }` — при $N \geq 4$ это уже отдельная программа со своими багами (порядок, начальные значения, distinct). В MVI комбинирование происходит до публикации — внутри Reducer.
3. **One-shot события без дома.** В «чистом» MVVM тост/навигация мечутся между `SingleLiveEvent`, каналами и флагами в state. В MVI у них есть формальный дом — `NEWS` (со своими проблемами доставки, см. Тему 8, но хотя бы с явным контрактом).
4. **Тесты на порядок вызовов.** MVVM-тест часто проверяет «сначала показали лоадер, потом контент» через `verify(order)`. В MVI это табличный тест чистой функции: `reducer(state, effect) == expected` — без моков и корутин.

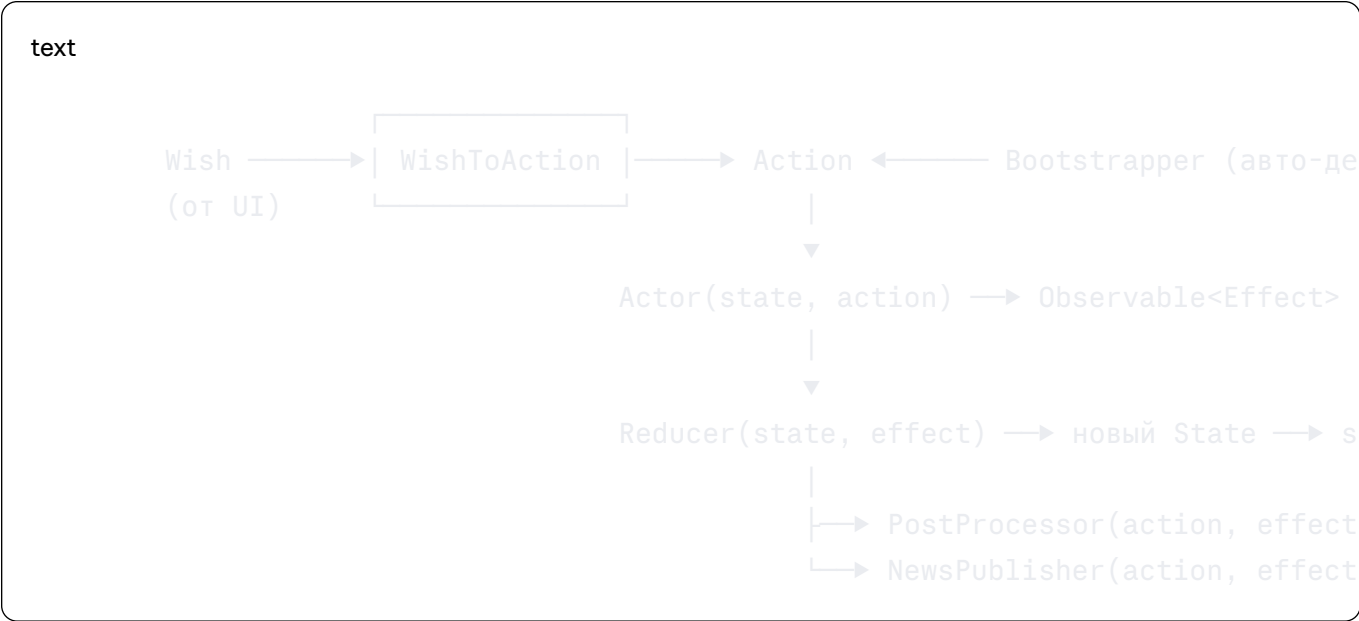
Честность для интервью: сами по себе «единый immutable state», «SSOT», «events up / state down» — это свойства UDF вообще, их даёт и дисциплинированный MVVM (`UiState` + `onAction()` + приватный reducer). Специфика именно MVI — формализация (`Actor`/`Effect`/`Reducer`/`News` как отдельные тестируемые сущности), сериализация редукции на уровне рантайма и, в случае MVICore, — middleware/time-travel tooling. Сильная фраза: «Если ViewModel принимает события и публикует единый state — это уже UDF; MVI добавляет формализованную state machine с гарантией последовательной редукции».

Тема 4. Канонический MVICore против вашего wrapper'a: полная карта соответствия

Аналогия. MVICore — полный конструктор железной дороги: рельсы, стрелки, семафоры, депо, диспетчерская с записью всех движений. Ваш wrapper — собранный из него один надёжный маршрут «туда-обратно»: ездит отлично, но стрелок и депо на карте нет. На собеседовании спросят именно про стрелки.

4.1. Анатомия полного `BaseFeature`

Каноническая сигнатура — семь компонентов и два вида входов:



Компонент	Контракт	Назначение
Wish	вход accept()	Публичное намерение извне (UI, deeplink)
WishToAction	Wish → Action	Трансляция во «внутренний язык» feature
Bootstrapper	() → Observable<Action>	Автозапуск действий при создании: initial load, подписка на репозиторий
Actor	(State, Action) → Observable<Effect>	Асинхронная работа, единственное место side-effect'ов «наружу»
Reducer	(State, Effect) → State	Чистая синхронная редукция
PostProcessor	(Action, Effect, State) → Action?	Порождение <i>следующих</i> действий после редукции — цепочки внутри feature
NewsPublisher	(Action, Effect, State) → News?	Одноразовые события; получает уже новое состояние

Порядок внутри цикла (важно для интервью): **Reducer** → публикация нового state → **PostProcessor** → **NewsPublisher**. То есть и post-processing, и news видят состояние **после** редукции.

4.2. Что взято, что отброшено

`ActorReducerFeature` — официальное упрощение `BaseFeature`, где `WishToAction = identity` (типы `Wish` и `Action` совпадают) и нет `PostProcessor`'а. Ваш `ActorReducerFeatureAdapter` строится именно на нём — и не передаёт даже опциональный `Bootstrapper`:

Канонический MVICore	В вашем wrapper'е	Последствие
<code>Wish</code> (внешнее намерение)	<code>ACTION</code>	<code>Wish</code> и <code>Action</code> схлопнуты — «внутреннего языка» нет
<code>Action</code> (внутренние действия)	—	Цепочки и авто-действия невозможны на уровне feature
<code>Bootstrapper</code>	отсутствует	Стартовую загрузку инициирует UI (<code>ScreenOpened</code>)
<code>Actor</code> (Rx)	<code>Actor</code> (Flow) + <code>ActorAdapter</code>	Контракт корутинный, рантайм — Rx (Тема 5)
<code>Effect</code>	<code>EFFECT</code>	1:1 — вход редьюсера, не UI-событие
<code>Reducer</code>	<code>Reducer</code>	1:1
<code>PostProcessor</code>	отсутствует	«после эффекта X сделай Y» — ручную
<code>NewsPublisher</code>	<code>NewsPublisher</code>	1:1, тоже видит новое состояние
<code>Feature.state</code> (Rx <code>BehaviorSubject</code>)	+ зеркало <code>MutableStateFlow</code>	Два контейнера состояния (см. 4.4)
<code>Feature.news</code> (Rx)	<code>SharedFlow</code> + своя очередь	Собственная политика доставки (Тема 8)
<code>Binder</code> + lifecycle	<code>viewModelScope</code> + <code>launchIn</code>	Централизованно в Android ViewModel
Middleware, logger, time-travel	не используются	Главный tooling-аргумент MVICore потерян; логировать — руками (Timber)
<code>FeatureScheduler</code>	всегда <code>null</code> в публичном конструкторе	Thread affinity = поток создания VM (Тема 5)

4.3. Три практических следствия отброшенного

Нет `Bootstrapper` → двойная ответственность UI. Загрузку запускает `acceptAction(ScreenOpened)` из UI. Риски: (а) до первого action feature «мертва» — подписки на репозитории не начаты; (б) при пересоздании view над живой ViewModel (`Fragment` recreate, возврат по back stack) `ScreenOpened` прилетит повторно. Отсюда обязательное правило: **Actor идемпотентен по отношению к стартовому action:**

kotlin

```
/** Замена отсутствующего Bootstrapper: повторный ScreenOpened над готовым конт  
private fun loadIfNeeded(state: BenefitsState): Flow<BenefitsEffect> =  
    if (state.isLoading || state.items.isNotEmpty()) emptyFlow() else load(stat
```

Нет `PostProcessor` → цепочки склеиваются в Actor. Сценарий «после успешного удаления перезагрузить список» в каноне: `NewsPublisher`-подобный `PostProcessor` вернул бы `Action.Reload`. У вас — либо один actor-flow эмитит обе фазы эффектов (`ItemDeleted`, затем эффекты перезагрузки), либо — анти-паттерн — UI ловит `News` и шлёт новый `Action` (state machine начинает зависеть от подписанного UI).

Нет middleware → нет time-travel и записи переходов. Это то, ради чего Badoo писали MVICore. Компенсация — ручное логирование в Actor/Reducer по вашему стандарту:

kotlin

```
Timber.tag(TAG).d("mylog 001: reduce effect=${effect::class.simpleName} -> stat
```

(обратите внимание: `effect::class.simpleName` в логе-тексте допустим, но `TAG` — только hardcoded-константа, R8!)

4.4. Два контейнера состояния и граница адаптера

Настоящий владелец состояния — `BehaviorSubject` внутри `BaseFeature`. `_state: MutableStateFlow` во ViewModel — его **асинхронное зеркало**, наполняемое коллектором в `viewModelScope`:

kotlin

```
feature.state // Rx → cold Flow через asFlow()  
    .onEach { _state.emit(it) } // зеркалирование  
    .catch { Timber.e(it) } // ⚠ мина — Тема 7  
    .launchIn(viewModelScope)
```

Следствия, которые проверяют на Senior-интервью:

- **Тесты:** сразу после `acceptAction()` значение `viewModel.state.value` может быть ещё старым — между `feature` и зеркалом лежит диспетчеризация корутины. Нужен `advanceUntilIdle()`/`runCurrent()` или сбор через `Turbine`, а лучше — тестировать `Reducer`/`Actor` напрямую (Тема 12).
- **Диагностика:** `feature` может уйти вперёд зеркала, а при смерти моста (Тема 7) — жить вечно без зеркала.
- **Reducer** меняет состояние `feature`, а не `MutableStateFlow`: `ViewModel` сама ничего не редуцирует.

4.5. Сопоставление с «эталонной» MVI-схемой и официальным Android UDF

Распространённая каноническая схема (Redux-подобная, её же используют `Orbit`/`MVIKotlin` в своих терминах):

text

Intent → Store → Middleware/Executor → Result/Mutation → Reducer → State → View
↳ Side Effect /

Канон

Ваш термин

Intent

`ACTION`

Store

`ActorReducerFeature` (+ `MviViewModel` как Android-host)

Middleware / Executor

`Actor`

Result / Mutation

`EFFECT`

Reducer

`Reducer`

State

`STATE`

Side Effect / Label / Event

`NEWS`

Концептуально это один и тот же паттерн; отличается **runtime**: у вас Rx-feature за Flow-фасадом, в современных coroutine-native реализациях — `CoroutineScope` + reducer loop поверх `Channel`/`MutableStateFlow.update`.

Позиция официального гайда Android (важно проговорить корректно): Google рекомендует не «MVI» и не «MVVM», а **UDF** — единый immutable `UiState` из state holder'a, события снизу вверх, lifecycle-aware collection. Официальный гайд также *предпочитает* превращать one-shot события ViewModel → UI в обновление state, а не слать произвольные event-потоки; это opinionated-рекомендация, и ваша `NEWS`-модель — осознанное отступление от неё, которое нужно уметь аргументировать (Тема 8 даёт аргументы обеих сторон).

Терминологическая ловушка на интервью: во многих coroutine-MVI кодовых базах словом *Effect* называют одноразовое UI-событие (то, что у вас `NEWS`). У вас `EFFECT` — это **вход редьюсера** (то, что в каноне Result/Mutation). Проговорите это в первые 30 секунд обсуждения архитектуры, иначе полдиалога уйдёт на рассинхрон словарей.

Тема 5. Кому принадлежит поток: `FeatureScheduler`, `SameThreadVerifier` и правда про `ActorAdapter`

Аналогия. У feature есть «домашний стадион» — поток, на котором её создали. Играть (делать запросы) можно на выезде, но судья (`SameThreadVerifier`) засчитывает голы (редукцию эффектов) только на домашнем поле. `FeatureScheduler` — это переезд клуба на собственную арену; у вас переезд запрещён уставом.

5.1. Модель ниток MVICore

- **Без `FeatureScheduler` (ваш случай):** feature привязана к потоку создания. `accept()` и применение эффектов должны происходить на нём; `SameThreadVerifier` при нарушении бросает исключение.
- **С `FeatureScheduler`:** обработка переносится на переданный scheduler; он обязан быть однопоточным, иначе feature недетерминирована.

Ваш публичный конструктор жёстко передаёт `null`:

```
kotlin
constructor(initialState, actor, reducer, newsPublisher) :
    this(initialState, actor, reducer, newsPublisher, /* featureScheduler = */
```

ViewModel создаются на main → де-факто **main-thread affinity**, но контракт нигде не выражен: `acceptAction` не помечен `@MainThread`. Вызов из фонового потока (callback SDK, push-обработчик) — готовый инцидент «экран завис, крэша нет».

5.2. Что на самом деле делает `ActorAdapter`

Подтверждённая деталь реализации: адаптер конвертирует `Flow<EFFECT>` актора так:


```
kotlin
```

```
flow.asObservable(Dispatchers.Main.immediate)
```

Три следствия — каждое стоит отдельного вопроса на интервью:

1. **Эмиссии приходят на main автоматически.** `asObservable(context)` собирает flow в переданном контексте; контекст коллектора определяет поток, на котором вызывается `onNext`. `flowOn(Dispatchers.IO)` внутри актора влияет только на upstream-работу — финальная доставка эффекта всё равно на `Main.immediate`. Поэтому KDoc-предупреждение «в конце цепочки переключитесь на UI-поток» — упрощение. Точная формулировка: **возвращать эмиссии нужно на поток, которому принадлежит feature; текущий адаптер делает это сам.** Реальная опасность остаётся только в `channelFlow`/`callbackFlow`, где `send()` вызывается из чужого потока колбэка, — но и там `asObservable` доставит в своём контексте. Практическое правило команды: тяжёлую работу — в `main-safe suspend`-функциях репозитория (`withContext(IO)` внутри), никаких Rx `observeOn` в акторах.
2. **Structured cancellation «из коробки».** `asObservable` вешает `setCancellable { job.cancel() }`; dispose Rx-подписки → отмена корутины → `CancellationException` пролетает через flow до suspend-вызова репозитория. Это ваша единственная нить отмены запросов — см. 6.4.
3. **FeatureScheduler заблокирован дважды.** Даже если пробросить фоновый single-thread scheduler во внутренний конструктор, эффекты продолжают приходить на main (адаптер!) → thread-check упадёт. Итог: «недоступен» — не оговорка, а следствие двух независимых решений (публичный конструктор + hardcode контекста в адаптере).

5.3. Что происходит при нарушении thread-контракта

Исключение `SameThreadVerifier` рождается внутри Rx-цепочки feature. Дальше два сценария:

- В приложении установлен глобальный `RxJavaPlugins.setErrorHandler` (в вашем проекте — установлен): ошибка уходит в него → лог → **внутренняя подписка мертва, процесс жив.** Отсюда фольклор из KDoc: «приложение не упадёт, но работать ожидаемо не будет». Feature перестаёт применять эффекты — «полуживой» экран.
- Глобального handler'а нет: RxJava 2 маршрутизирует недоставленную ошибку как `UndeliverableException` в дефолтный обработчик → на Android это, как правило, краш процесса.

Вывод для интервью: «не упадёт» — это свойство *конфигурации проекта* (глобальный handler), а не гарантия MVICore.

5.4. Минимальные фиксы

kotlin

```
// MviViewModel
@MainThread
open fun acceptAction(action: ACTION) {
    if (BuildConfig.DEBUG) {
        check(Looper.getMainLooper().isCurrentThread) {
            "acceptAction must be called on main thread, was: ${Thread.currentThreadName}"
        }
    }
    feature.accept(action)
}
```

Долгосрочные варианты (по нарастанию стоимости): явный single-thread

`FeatureScheduler` + переписанный `ActorAdapter`

(`asObservable(scheduler.asCoroutineDispatcher())`); либо coroutine-native reducer loop, снимающий thread-affinity как класс проблем (Тема 12 части «стратегии» и финальный чек-лист).

Тема 6. Конкурентность Actor'ов: out-of-order, stale results и отмена

Аналогия. Кухня с одним окном выдачи и несколькими поварами. Окно (Reducer) выдаёт блюда строго по одному — но повара (подписки на Actor'ы) готовят параллельно, и стейк, заказанный первым, легко выйдет после салата, заказанного вторым.

6.1. Как именно `BaseFeature` исполняет Actor'ы

По исходнику: на каждый `action` вызывается `actor(state, action)`, на возвращённый `Observable` создаётся **отдельная подписка**, все подписки складываются в общий `CompositeDisposable`, эффекты редуцируются по мере поступления. Ни `concatMap` (сериализация запросов), ни `switchMap` (отмена предыдущего), ни `exhaustMap` (игнор новых) — **plain parallel merge**:

text

```
t0 Action A → Actor A (медленный запрос) → Effect(ResultA)
t1 Action B → Actor B (быстрый запрос) → Effect(ResultB)
```

Порядок редукции: ResultB, затем ResultA ← последним словом останется УСТАРЕВШИЙ

Почему так спроектировано: `concatMap` заблокировал бы весь конвейер одним медленным запросом (в т.ч. независимые действия вроде аналитики), `switchMap` глобально — отменял бы независимые операции. MVICore отдаёт политику конкурентности на откуп фиче. У вас этой политики пока нет — надо добавить свою.

6.2. Stale snapshot: тонкость, которую все путают

`Actor` получает **снимок** состояния на момент обработки action. `Reducer` же всегда получает **актуальное** состояние из `stateSubject.value` на момент прихода эффекта. То есть редукция не применяется к устаревшему объекту состояния — но сам эффект мог быть *вычислен* по устаревшему снимку:

text

```
state.filter = "A" → Action.Load → Actor стартует запрос(filter="A")
state.filter = "B" (пользователь переключил) → Action.Load → быстрый ответ для
... приходит ответ для "A" → Reducer получает свежий state(filter="B"), но кладёт
```

6.3. Защита №1 (рекомендуемая): `requestId`/generation

Идея: каждый запуск загрузки получает монотонный идентификатор; результат с чужим идентификатором `Reducer` игнорирует. Для этого паттерна удобнее `data-class`-состояние с флагами (`sealed`-вариант потребует таскать `requestId` в каждом подклассе — шумно; выбор формы состояния — прагматика, не догма):

kotlin

```
/** Состояние экрана бенефитов */
internal data class BenefitsState(
    val requestId: Long = 0L,
    val isLoading: Boolean = false,
    val items: List<Benefit> = emptyList(),
    val error: BenefitsError? = null,
)

internal sealed interface BenefitsEffect {
    /** Начата загрузка поколения [requestId] */
    data class LoadingStarted(val requestId: Long) : BenefitsEffect
    /** Успешный результат поколения [requestId] */
    data class ContentLoaded(val requestId: Long, val items: List<Benefit>) : BenefitsEffect
    /** Ожидаемая ошибка поколения [requestId] */
    data class LoadingFailed(val requestId: Long, val error: BenefitsError) : BenefitsEffect
}
```

kotlin

```
internal class BenefitsReducer : Reducer<BenefitsState, BenefitsEffect> {

    override fun invoke(state: BenefitsState, effect: BenefitsEffect): BenefitsState {
        when (effect) {
            is BenefitsEffect.LoadingStarted -> state.copy(
                requestId = effect.requestId,
                isLoading = true,
                error = null,
            )
            is BenefitsEffect.ContentLoaded ->
                if (effect.requestId != state.requestId) state // ← stale guard
                else state.copy(isLoading = false, items = effect.items)
            is BenefitsEffect.LoadingFailed ->
                if (effect.requestId != state.requestId) state // ← stale guard
                else state.copy(isLoading = false, error = effect.error)
        }
    }
}
```

Генерация поколения — в Actor'e от снимка: `val requestId = state.requestId + 1`. Пока `acceptAction` строго main-thread (Тема 5), первый эффект `LoadingStarted` успевает отредуцироваться до следующего `accept`, и коллизий поколений нет; если main-контракт не закреплён — берите `AtomicLong` в Actor'e.

6.4. Защита №2: настоящая отмена — и почему её сейчас нет

Честный ответ для интервью: в текущем рантайме **отменить предыдущий запрос точно нельзя** — все actor-подписки живут в одном `CompositeDisposable`, который освобождается только целиком в `feature.dispose()`. Что есть:

- **Отмена при закрытии экрана работает полностью:** `onCleared()` → `feature.release()` → `dispose()` → Rx cancellable y `asObservable` → `Job.cancel()` → `CancellationException` доходит до suspend-репозитория. Два независимых механизма: `viewModelScope` гасит *мосты* (state/news коллекторы), `feature.release()` гасит *внутренности* feature. Ответ «viewModelScope всё отменит» — неверный: он не властен над подписками внутри MVICore.
- **Семантика `switchMap` для конкретной загрузки** достижима тремя путями, по нарастанию честности: (а) `requestId`-guard — результат придёт, но будет проигнорирован (сеть потрачена, состояние корректно); (б) внутренний координатор в Actor'e с собственным `Job` — работает, но заводит скрытое mutable-состояние в Actor'e (тот самый анти-паттерн, который у вас уже встречался — выносите такие поля хотя бы в явный `RequestCoordinator`); (в) coroutine-native runtime с `collectLatest`/`mapLatest` — системное решение.

6.5. Мини-чеклист конкурентности для code review

- Каждый «загрузочный» flow эмитит `LoadingStarted(requestId)` первым эффектом.
 - Reducer игнорирует чужие поколения для **всех** терминальных эффектов (успех и ошибка).
 - Стартовый action идемпотентен (см. 4.3).
 - В Actor'е нет mutable-полей без явного владельца-координатора.
 - `CancellationException` нигде не проглатывается (Тема 7).
-

Тема 7. Модель ошибок: что на самом деле значит «must never throw»

Аналогия. «В операционной не кричать» не означает «отключить пожарную сигнализацию». Врач не кричит на пациента (ожидаемые ошибки → аккуратный `EFFECT`), но если горит проводка — сигнализация обязана сработать (программный дефект → fail-fast и репорт), иначе сгорит вся клиника молча.

7.1. Откуда взялся контракт

У `BaseFeature` **нет error-канала**: state и news — отдельные subjects, ошибки Actor'а в них не маршрутизируются. Если actor-Observable завершится `onError`, внутренняя подписка feature умрёт, а ошибка уйдёт в глобальный `RxJavaPlugins.onError` (с установленным в проекте handler'ом — просто в лог). Итог необработанного throw — не краш, а **молчаливо переставшая работать feature**. Контракт «never throw» — это защита от этого сценария, а не эстетика.

7.2. Три класса ошибок — три разные политики

Класс	Примеры	Политика
Ожидаемые (domain/infra)	<code>IOException</code> , HTTP-коды, таймаут, пустые данные, отказ permission	Материализовать в <code>EFFECT</code> (<code>LoadingFailed</code>) — это часть state machine
Отмена	<code>CancellationException</code>	Всегда пробрасывать. Проглотить = сломать structured cancellation: скоуп продолжит считать корутину живой, <code>runCatching</code> вокруг suspend-вызова — классическая мина
Программные	невозможный <code>when</code> , <code>ClassCastException</code> , нарушенный invariant, кривой DI	Fail-fast + наблюдаемость: лог уровня fatal, crash reporting; молчаливый <code>return emptyFlow()</code> превращает дефект в silent corruption

Эталонный скелет Actor'a по всем правилам сразу (идемпотентность из 4.3, `requestId` из 6.3, классы ошибок, house-style логи):

kotlin

```
internal class BenefitsActor @Inject constructor(
    private val repository: BenefitsRepository,
) : Actor<BenefitsState, BenefitsAction, BenefitsEffect> {

    override fun invoke(action: BenefitsAction, state: BenefitsState): Flow<BenefitsEffect> =
        when (action) {
            BenefitsAction.ScreenOpened -> loadIfNeeded(state)
            BenefitsAction.RetryClicked -> load(state)
            is BenefitsAction.BenefitClicked -> flowOf(BenefitsEffect.BenefitSelected)
        }

    /** Идемпотентная замена Bootstrapper: повторный вход не перезапускает загрузку */
    private fun loadIfNeeded(state: BenefitsState): Flow<BenefitsEffect> =
        if (state.isLoading || state.items.isNotEmpty()) emptyFlow() else load(state)

    private fun load(state: BenefitsState): Flow<BenefitsEffect> = flow {
        val requestId = state.requestId + 1
        emit(BenefitsEffect.LoadingStarted(requestId))
        try {
            val items = repository.loadBenefits() // main-safe suspend, IO внутри
            emit(BenefitsEffect.ContentLoaded(requestId, items))
        } catch (ce: CancellationException) {
            throw ce // отмена — не ошибка
        } catch (io: IOException) {
            Timber.tag(TAG).e("mylog 001: Benefits load failed: ${io.message}")
            emit(BenefitsEffect.LoadingFailed(requestId, io.toBenefitsError()))
        }
        // Всё остальное НЕ ловим: программный дефект должен быть виден,
        // а не превращаться в вечный спиннер.
    }

    private companion object {
        private const val TAG = "BenefitsActor" // hardcode: R8 обфусцирует символы
    }
}
```

7.3. «Reducer может вернуть state вместо throw» — опасная половина правды

Для ожидаемой бизнес-ситуации — да, вернуть корректное состояние. Но для невозможной комбинации `state × effect` возврат старого state скрывает дефект и оставляет UI в неопределённости. Правильный инструмент — sealed-типы + исчерпывающий `when` (компилятор сам не даст «забыть» ветку), а для действительно невозможного — ``error("Unexpected effect=effect for state =state")``: в debug падаем сразу, в release это поймает crash reporting через глобальный Rx handler.

7.4. `catch { Timber.e(th) }` на мостах: анатомия «полуживой» ViewModel

Оператор `catch` завершает flow после обработки ошибки. Первая же ошибка в мосте `feature.state → _state` даёт конфигурацию:

text

feature жива → acceptAction принимается → Actor работает → state внутри feature
но
мост мёртв → StateFlow заморожен → Compose рисует последний снимок вечно → поль

Ошибка при этом существует только в логах. То же — для news-моста. Улучшенный мост делает отказ наблюдаемым:

kotlin

```
feature.state
    .onEach { newState -> _state.emit(newState) }
    .catch { th ->
        Timber.tag(TAG).e(th, "mylog 002: State bridge terminated")
        crashReporter.reportNonFatal(th) // отказ виден мониторингу
        if (BuildConfig.DEBUG) throw th // в debug — падаем: дефект должен б
    }
    .launchIn(viewModelScope)
```

Нюанс для интервью: этот `catch` не ловит исключения Actor'a — они живут во внутренней подписке feature (см. 7.1) и до моста не доходят. Фраза «приложение не упадёт, потому что снаружи есть catch» — ложная гарантия.

Тема 8. News под микроскопом: аудит never-drop-очереди и чем её заменить

Закрываем пункт 6 из Key Findings части 1/2: «never-drop-очередь» была названа архитектурным решением с трейд-оффом — вот весь трейд-офф.

Аналогия. Консьерж (`sendNews`) смотрит, дома ли жилец (`subscriptionCount`): дома — вручает письмо лично (`emit`), нет — кладёт в безразмерный ящик (`LinkedList`).

Проблемы: жилец мог выйти ровно в момент вручения (гонка); письма из ящика достают по одному, и если консьержа отозвали на полпути — письмо уже вынуто, но не вручено (потеря); в квартире может жить несколько человек — каждый получит копию (broadcast); а письмо «зайдите за посылкой» двухмесячной давности вручается как свежее (нет TTL).

8.1. Точная семантика каждой детали

`MutableSharedFlow()` по умолчанию = `replay = 0`, `extraBufferCapacity = 0`: при живых коллекторах `emit` *подвешивается*, пока все не примут значение; при нуле коллекторов `emit` завершается мгновенно, значение **тихо теряется**. Очередь и была придумана, чтобы закрыть второй случай.

Гонка check-then-emit. Проверка и эмиссия — не атомарны:

text

1. `subscriptionCount.value == 1` → выбираем ветку `emit`
2. коллектор отменяется (уход с экрана)
3. `_news.emit(news)` — подписчиков уже нет
4. `emit` мгновенно «успешно» завершается → событие потеряно навсегда

`subscriptionCount` — счётчик для оптимизаций producer'a, а не транзакционный замок доставки.

`poll()` до завершения `emit` в `dispatchQueue`. Элемент удаляется из очереди, затем корутина подвисает на `emit`; отмена в этом окне = элемент уже вынут, но не доставлен, повтора не будет.

Инверсия порядка. Пока `dispatchQueue` выгружает A и B, живой `sendNews(C)` может эмитнуть напрямую — коллектор увидит порядок, не совпадающий с порядком генерации.

Broadcast. `SharedFlow` — широковещательный: если news соберут экран, и Activity, и analytics-слой — каждый обработает событие. «Ровно один получатель» текущая система не обещает.

Память. `LinkedList` без верхней границы: повторяющиеся ошибки/прогресс от фоновых источников копят, пока экран не подписан.

Configuration change — единственный выигрыш. Rotation: старый коллектор исчез → короткое окно `subscriptionCount == 0` → события в очередь → новый экран подписался → очередь выгружена. Ради этого всё и затевалось, и это работает.

Возврат на экран — обратная сторона. Пока пользователь был в другом месте, ViewModel жила и копила news; при возвращении он получает пачку устаревших snackbar'ов, а хуже — **повторную навигацию или повторный внешний intent**. Без TTL/ID/acknowledgement система не отличает «свежее» от «протухшего».

Process death. Очередь и state — только в памяти: новая ViewModel стартует с `initialState`, недоставленные события исчезают.

8.2. Вердикт по гарантиям

Гарантия	Есть?	Почему нет
at-most-once	✗	очередь + пересоздание экрана могут доставить событие «ещё раз» с точки зрения пользователя
at-least-once	✗	гонка check-then-emit; потеря при отмене dispatch
exactly-once	✗	следует из двух строк выше
Итог	—	best-effort broadcast + неограниченный in-memory буфер для окон без подписчика

В типовом сценарии «одна ViewModel — один последовательно переподключающийся screen-collector» система *обычно* ведёт себя как ожидается. Формальных гарантий — нет. Это и есть честная формулировка для интервью.

8.3. Чем заменять: карта решений

Вариант	Гарантия	Rotation	Process death	Cor
A. Durable state + ack ((pendingEffect) в STATE, UI шлёт EffectHandled(id))	ближе всех к exactly-once	✓	✓ с SavedStateHandle	1 (я)
B. Channel(BUFFERED).receiveAsFlow()	буферизуется без коллектора; каждый event — ровно одному	✓	✗	1 (u
C. Bounded SharedFlow(extraBufferCapacity = N, DROP_OLDEST)	при нуле подписчиков всё ещё теряет	✓ частично	✗	bro:
D. Persistent queue (Room + статус)	at-least-once + аудит	✓	✓	упр
E. Navigation coordinator	зависит от реализации	✓	зависит	1

Drop-in вариант В внутри MviViewModel (меняются только показанные члены; семантика меняется с broadcast на unicast — проверьте, что второго коллектора news в проекте нет):

kotlin

```
private val newsChannel: Channel<NEWS> = Channel(
    capacity = NEWS_BUFFER_CAPACITY,
    onBufferOverflow = BufferOverflow.DROP_OLDEST,
)

val news: Flow<NEWS> = newsChannel.receiveAsFlow()

private fun sendNews(news: NEWS) {
    val result = newsChannel.trySend(news)
    if (result.isFailure) {
        Timber.tag(TAG).e("mylog 003: News dropped: $news")
    }
}

private companion object {
    private const val TAG = "MviViewModel"
    private const val NEWS_BUFFER_CAPACITY = 64
}
```

Что это чинит: гонку check-then-emit (буфер работает и без коллектора), потерю в dispatch (нет ручной очереди), unbounded-память (ёмкость + политика). Что осознанно меняет: DROP_OLDEST вместо never-drop (для UI-событий старое и должно умирать) и unicast вместо broadcast. Что НЕ чинит: replay устаревшей навигации при возврате и process death — для них вариант А:

kotlin

```
/** Навигация как часть состояния: переживает пересоздание, требует подтвержден
internal data class BenefitsState(
    // ...
    val pendingNavigation: PendingNavigation? = null,
)

internal data class PendingNavigation(val id: Long, val target: BenefitsTarget)

// UI (Fragment): выполнил переход → подтвердил
viewModel.acceptAction(BenefitsAction.NavigationHandled(pending.id))
// Reducer: сбрасывает pendingNavigation, если id совпал
```

Тема 9. Lifecycle-границы: rotation, уход с экрана, process death

9.1. Три сценария — три разных судьбы state machine

Сценарий	ViewModel	Feature + Actor'ы	STATE	News-очередь
Rotation / config change	живёт	живут, запросы продолжаются	сохраняется	сохраняется (и выгрузится новому экрану)
Уход с экрана (VM жива, view нет)	живёт	живут	сохраняется	копится без TTL (Тема 8)
Process death	пересоздаётся	пересоздаются	сброс в <code>initialState</code>	испаряется

Частая ошибка на интервью — смешивать первый и третий: `ViewModel` переживает конфигурационные изменения, но **не** переживает system-initiated process death. Проверять третий сценарий руками: «Don't keep activities» недостаточно — нужен настоящий kill (`adb shell am kill <package>` из background).

9.2. `SavedStateHandle`: что сохранять и как это дружит с pure Reducer

Правило платформы: в `SavedStateHandle` — лёгкий transient-ввод (ID, фильтры, введённый текст), а не весь `STATE`; большие данные после пересоздания перезагружаются из repository. Для MVI это значит: сохраняем **recovery input**, из которого Actor восстановит остальное.

Тонкость, которую почти никто не проговаривает: `Reducer` — чистая функция, писать в `SavedStateHandle` из него нельзя. Правильное место — подписка на state внутри ViewModel:

kotlin

```
internal class BenefitsViewModel(  
    private val savedInstanceState: SavedStateHandle,  
    actor: BenefitsActor,  
) : MviViewModel<BenefitsAction, BenefitsEffect, BenefitsState, BenefitsNews>(  
    initialState = BenefitsState(  
        selectedCategoryId = savedInstanceState[KEY_SELECTED_CATEGORY], // восста  
    ),  
    actor = actor,  
    reducer = BenefitsReducer(),  
    newsPublisher = BenefitsNewsPublisher(),  
) {  
  
    init {  
        state  
            .map { benefitsState -> benefitsState.selectedCategoryId }  
            .distinctUntilChanged()  
            .onEach { categoryId -> savedInstanceState[KEY_SELECTED_CATEGORY] = c  
            .launchIn(viewModelScope)  
    }  
  
    private companion object {  
        private const val KEY_SELECTED_CATEGORY = "selected_category_id"  
    }  
}
```

9.3. Dagger 2 без Hilt: фабрика с `SavedStateHandle`

Раз в проекте ручная инъекция, `SavedStateHandle` заводится через `AbstractSavedStateViewModelFactory`:

kotlin

```
internal class BenefitsViewModelFactory @Inject constructor(  
    private val actorProvider: Provider<BenefitsActor>,  
) {  
  
    fun create(owner: SavedStateRegistryOwner): ViewModelProvider.Factory =  
        object : AbstractSavedStateViewModelFactory(owner, null) {  
            override fun <T : ViewModel> create(  
                key: String,  
                modelClass: Class<T>,  
                handle: SavedStateHandle,  
            ): T {  
                @Suppress("UNCHECKED_CAST")  
                return BenefitsViewModel(  
                    savedStateHandle = handle,  
                    actor = actorProvider.get(),  
                ) as T  
            }  
        }  
}
```

Во фрагменте: `private val viewModel: BenefitsViewModel by viewModels {
viewModelFactory.create(this) }` после `injector.inject(this)`.

Тема 10. Widget-иерархия: unchecked cast и второй вход в state machine

10.1. В чём дефект

kotlin

```
@Suppress("UNCHECKED_CAST")  
override fun acceptAction(action: WidgetAction) = super.acceptAction(action as
```

Интерфейс `WidgetViewModel` обещает принимать любой `WidgetAction`; конкретная реализация умеет только свой подтип. Компилятор проверить не может — из-за `type erasure` каст «срабатывает» молча, а `ClassCastException` взрывается позже и в другом месте: внутри Actor'a/Reducer'a при первом реальном обращении к полям, со стектрейсом, не указывающим на виновника. Это учебный пример того, почему `@Suppress("UNCHECKED_CAST")` в базовом классе фреймворка — долг, а не решение.

10.2. Фикс: generic-контракт

kotlin

```
/** Обязательный интерфейс ViewModel виджета */
interface WidgetViewModel<ACTION : WidgetAction> {
    val state: StateFlow<WidgetState> // заодно сужаем Flow → StateFlow
    fun acceptAction(action: ACTION)
    suspend fun onCommand(command: Command)
}

abstract class WidgetMviViewModel<ACTION : WidgetAction, EFFECT : WidgetEffect,
    actor: Actor<WidgetState, ACTION, EFFECT>,
    reducer: Reducer<WidgetState, EFFECT>,
    newsPublisher: NewsPublisher<ACTION, EFFECT, WidgetState, NEWS>,
> : MviViewModel<ACTION, EFFECT, WidgetState, NEWS>(
    initialState = WidgetState.Initial,
    actor = actor,
    reducer = reducer,
    newsPublisher = newsPublisher,
), WidgetViewModel<ACTION> {

    /** Единственная точка превращения внешних команд во внутренние Action */
    protected abstract fun mapCommandToAction(command: Command): ACTION?

    override suspend fun onCommand(command: Command) {
        mapCommandToAction(command)?.let(::acceptAction)
    }
}
```

Если widget-фреймворк жёстко требует негенерик `WidgetViewModel` — не прячьте каст в базовый класс: сделайте явный адаптер с валидацией (`actionClass.isInstance(action)`), иначе лог + по-оп) на границе фреймворка, чтобы отказ был локализован и наблюдаем.

10.3. `onCommand` — почему это архитектурная проблема, а не мелочь

`onCommand` — **второй вход** в state machine помимо `acceptAction`. Если реализация напрямую мутирует состояние, дёргает репозиторий или публикует UI-событие — UDF сломан: появился путь изменения состояния мимо Reducer'a, невозпроизводимый и нетестируемый табличными тестами. Правило: любой внешний вход (команда фреймворка, deeplink, push) конвертируется в `ACTION` — ровно это и закрепляет `mapCommandToAction` выше. То же самое относится к трём императивным колбэкам `MiniWidgetViewModel` (`onLoadMiniWidgetContent`/`onShowMiniWidgetContent`/`onCancelLoading`): в MVI-наследниках их тела должны сводиться к `acceptAction(...)`.

10.4. Бонус-вопрос интервью: наследование vs композиция

Цепочка `ConcreteWidgetVM → WidgetMviViewModel → MviViewModel → ViewModel` протаскивает четыре generic-параметра через всю иерархию, а базовый класс владеет сразу lifecycle, state-мостом и news-политикой — заменить одно без другого нельзя. Композиционная альтернатива — `MviFeatureHost<ACTION, STATE, NEWS>` (feature + мосты + release), который ViewModel просто держит полем. Выигрыш: тестируемость хоста отдельно от Android, отсутствие каст-акробатики, и главное — возможность подменить Rx-runtime на coroutine-native, не трогая наследников. Это ключевой ход стратегии «изолировать Rx» из финального чек-листа.

Тема 11. Интеграция с Compose в вашем стеке (Fragment + ComposeView + FragmentAttacher)

11.1. Полный паттерн экрана

kotlin

```
internal class BenefitsFragment : Fragment() {

    @Inject
    lateinit var viewModelFactory: BenefitsViewModelFactory

    @Inject
    lateinit var detailsApi: BenefitDetailsApi

    private val viewModel: BenefitsViewModel by viewModels { viewModelFactory.c

    override fun onCreate(savedInstanceState: Bundle?) {
        injector.inject(this) // ручной Dagger 2 – до super
        super.onCreate(savedInstanceState)
    }

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?,
    ): View = ComposeView(requireContext()).apply {
        // Критично для фрагментов: композиция должна умирать вместе с VIEW-lif
        // а не с самим ComposeView – иначе утечки состояния при detach/attach.
        setViewCompositionStrategy(ViewCompositionStrategy.DisposeOnViewTreeLif
        setContent {
            AppTheme {
                BenefitsRoute(viewModel = viewModel)
            }
        }
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        // News собираем во фрагменте: он ближе всего к FragmentAttacher.
        viewLifecycleOwner.lifecycleScope.launch {
            viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
                viewModel.news.collect(::handleNews)
            }
        }
        viewModel.acceptAction(BenefitsAction.ScreenOpened)
    }

    private fun handleNews(news: BenefitsNews) {
        when (news) {
            is BenefitsNews.OpenBenefitDetails ->
                mainAttacher.pushToStack(
```

```

        detailsApi.getPresentationEntry(news.id),
        BenefitDetailsApi.DETAILS_TAG,
    )
    is BenefitsNews.ShowError -> showSnackbar(news.message)
}
}
}

```

kotlin

```

/** Stateful-обёртка: единственное место, знающее о ViewModel */
@Composable
internal fun BenefitsRoute(viewModel: BenefitsViewModel) {
    val benefitsState by viewModel.state.collectAsStateWithLifecycle()

    BenefitsScreen(
        state = benefitsState,
        onAction = viewModel::acceptAction, // method reference — стабильная сс
    )
}

/** Stateless-экран: только рендер, идеально превьюится и тестируется */
@Composable
private fun BenefitsScreen(
    state: BenefitsState,
    onAction: (BenefitsAction) -> Unit,
) {
    // Render only.
}

```

11.2. Почему именно так

collectAsStateWithLifecycle vs **collectAsState**. Первый — рекомендованный Android-способ: останавливает коллекцию вне **STARTED** (по умолчанию), т.е. в фоне мост «Compose ← StateFlow» не жжёт работу и не держит upstream. Второй — platform-agnostic и собирает всегда, пока композиция жива. Для **StateFlow** состояния разница по корректности невелика (conflation спасает), но lifecycle-вариант — прямая рекомендация и правильный ответ на интервью.

News ≠ state, поэтому ни тот ни другой. **collectAsState*** по определению даёт «последнее значение как состояние» — событие при этом либо потеряется (conflation), либо повторится при каждой новой подписке. Событиям нужен *активный однократный* сбор: `repeatOnLifecycle(STARTED) { news.collect(...) }`. Если событие должно жить внутри композиции (например, `SnackbarHostState.showSnackbar`), тот же паттерн внутри Compose:

kotlin

```
val lifecycleOwner = LocalLifecycleOwner.current
LaunchedEffect(viewModel, lifecycleOwner) {
    lifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
        viewModel.news.collect { news -> /* SnackbarHostState.showSnackbar(...) */
    }
}
```

Связка с Темой 8, которую проверяют дотошные интервьюеры: `STARTED`-коллекция означает, что в фоне подписчика нет → ваша очередь (или Channel-буфер) копит события → при возврате они воспроизведутся пачкой. Для снейбаров терпимо, для навигации — нет: ещё один аргумент за durable state + ask (вариант А) именно для навигационных событий.

`ScreenOpened` из `onViewCreated` прилетит при каждом пересоздании view над живой VM — тот самый случай, ради которого Actor идемпотентен (4.3). Проговорите эту пару (вызов + guard) как единое решение.

11.3. Стабильность STATE для рекомпозиции

Мост в первую часть серии: `STATE` пересекает границу модулей, поэтому его stability для Compose-компилятора — контрактный вопрос. Практический минимум: `immutable` (`data class`); коллекции — `kotlinx.collections.immutable` (`ImmutableList`) или аннотация `@Immutable` на обёртке; колбэки вниз — `method reference` (`viewModel::acceptAction`), а не свежая лямбда с захватом. Strong skipping (по умолчанию с Kotlin 2.0.20) смягчает цену unstable-параметров и мемоизирует лямбды, но контрактная стабильность надёжнее «надежды на компилятор» — особенно в многомодульном проекте, где feature-модуль скомпилирован отдельно.

Тема 12. Тестовая стратегия: JUnit5 + MockK

12.1. Пирамида именно для этой архитектуры

- **~70% — табличные тесты (Reducer)**. Чистая синхронная функция: ни моков, ни корутин, ни Android. Самые дешёвые и самые ценные — они и есть исполняемая спецификация переходов состояния. Именно их вы сохраняете нетронутыми при любой будущей миграции runtime.
- **~20% — тесты (Actor)**. `Flow<EFFECT>` собирается через `toList()` внутри `runTest`; зависимости — MockK. Проверяем: последовательность эффектов, `requestId`-поколения, превращение ожидаемых ошибок в эффекты, проброс отмены.
- **~10% — smoke-тесты (MviViewModel)**. Это интеграционные тесты через Rx-мост и main-диспетчер: медленные и хрупкие по построению. Гоняем на *связность* (action дошёл до state), а не на ветвления бизнес-логики — ветвления уже покрыты уровнями выше.

Логика распределения — прямое следствие Темы 4.4: между feature и `StateFlow` лежит асинхронная граница адаптера, поэтому «тестировать всё через ViewModel» означает тестировать мост вместо логики.

12.2. Инфраструктура: `MainDispatcherExtension` для JUnit5

Feature создаётся на потоке теста и требует main (Тема 5), а `ActorAdapter` собирает flow на `Dispatchers.Main.immediate` — значит, `Main` в тестах обязан быть подменён:

```
kotlin

/** Подменяет Dispatchers.Main на тестовый диспетчер для каждого теста */
@OptIn(ExperimentalCoroutinesApi::class)
class MainDispatcherExtension(
    val dispatcher: TestDispatcher = UnconfinedTestDispatcher(),
) : BeforeEachCallback, AfterEachCallback {

    override fun beforeEach(context: ExtensionContext) {
        Dispatchers.setMain(dispatcher)
    }

    override fun afterEach(context: ExtensionContext) {
        Dispatchers.resetMain()
    }
}
```

`UnconfinedTestDispatcher` — чтобы «immediate»-эмиссии адаптера выполнялись без ручного прокручивания; когда в тесте важен контроль времени — локально берите `StandardTestDispatcher` и двигайте виртуальное время сами.

12.3. `Reducer`: табличные тесты, включая stale guard

kotlin

```
internal class BenefitsReducerTest {

    private val reducer = BenefitsReducer()

    @Test
    fun `content loaded for current request updates state`() {
        val state = BenefitsState(requestId = 42L, isLoading = true)
        val effect = BenefitsEffect.ContentLoaded(requestId = 42L, items = benefits)

        val result = reducer(state, effect)

        assertEquals(
            BenefitsState(requestId = 42L, isLoading = false, items = benefits),
            result,
        )
    }

    @Test
    fun `stale content loaded is ignored`() {
        val state = BenefitsState(requestId = 2L, isLoading = true)
        val staleEffect = BenefitsEffect.ContentLoaded(requestId = 1L, items = benefits)

        assertEquals(state, reducer(state, staleEffect)) // ← защита из Темы 6
    }
}
```

Когда переходов много — `@ParameterizedTest` + `@MethodSource` с тройками `(state, effect, expected)`: таблица переходов читается как документация.

12.4. `Actor`: успех, ожидаемая ошибка, отмена

kotlin

```
internal class BenefitsActorTest {

    private val repository = mockk<BenefitsRepository>()
    private val actor = BenefitsActor(repository)

    @Test
    fun `retry emits started then content with next generation`() = runTest {
        coEvery { repository.loadBenefits() } returns benefits
        val state = BenefitsState(requestId = 7L)

        val effects = actor(BenefitsAction.RetryClicked, state).toList()

        assertEquals(
            listOf(
                BenefitsEffect.LoadingStarted(requestId = 8L),
                BenefitsEffect.ContentLoaded(requestId = 8L, items = benefits),
            ),
            effects,
        )
    }

    @Test
    fun `expected error becomes effect not exception`() = runTest {
        coEvery { repository.loadBenefits() } throws IOException("boom")

        val effects = actor(BenefitsAction.RetryClicked, BenefitsState()).toList()

        assertTrue(effects.last() is BenefitsEffect.LoadingFailed)
    }

    @Test
    fun `cancellation propagates and cancels repository call`() = runTest {
        val requestStarted = CompletableDeferred<Unit>()
        coEvery { repository.loadBenefits() } coAnswers {
            requestStarted.complete(Unit)
            awaitCancellation() // репозиторий «висит», пока его не отменят
        }

        val collectJob = launch { actor(BenefitsAction.RetryClicked, BenefitsState(), requestStarted).await() }
        collectJob.cancelAndJoin()

        assertTrue(collectJob.isCancelled) // CancellationException не был прог
    }
}
```

Третий тест — исполняемая проверка контракта из Темы 7: если кто-то обернёт тело `load` в `runCatching` без `rethrow`, тест зависнет/упадёт — дефект виден сразу.

12.5. Smoke-тест `MviViewModel`: помним про асинхронное зеркало

kotlin

```
@ExtendWith(MainDispatcherExtension::class)
internal class BenefitsViewModelTest {

    private val repository = mockk<BenefitsRepository>()

    @Test
    fun `screen opened loads content into state`() = runTest {
        coEvery { repository.loadBenefits() } returns benefits
        val viewModel = BenefitsViewModel(actor = BenefitsActor(repository)) //

        viewModel.acceptAction(BenefitsAction.ScreenOpened)
        advanceUntilIdle() // ← обязательный шаг: Rx-мост + зеркало StateFlow а

        assertEquals(benefits, viewModel.state.value.items)
    }
}
```

Проверка `state.value` сразу после `acceptAction` без `advanceUntilIdle()` — флаки-тест по построению (Тема 4.4). Если такой тест у вас «стабильно зелёный» — это случайность планировщика, а не гарантия.

12.6. Чек-лист интеграционных News-сценариев

Если оставляете текущую очередь (или вводите `Channel`) — вот сценарии, каждый из которых заслуживает теста, потому что каждый — реальный баг-репорт из Темы 8:

- коллектор отменяется между проверкой `subscriptionCount` и `emit` (событие не должно теряться);
 - rotation: окно без подписчика → очередь → выгрузка новому коллектору;
 - уход с экрана и возврат: что происходит с накопленными событиями (TTL/ack-политика);
 - два одновременных коллектора (broadcast vs unicast — какое поведение вы обещаете);
 - отмена во время `dispatchQueue` (судьба уже `poll`-нутого элемента);
 - порядок queued- и live-событий;
 - переполнение bounded-буфера (что именно дропается и логируется ли);
 - пересоздание после process death (ничего не должно «воскресать» некорректно).
-

Interview Prep: вопросы, модельные ответы, ловушки

Формат каждого вопроса: **сильный ответ** (как отвечает Senior — 2-4 предложения по делу) → **типичная ошибка** (что говорят кандидаты, которых не берут) → **follow-up** (куда интервьюер копнёт дальше).

Блок А. Архитектура и словарь

Q1. «У вас MVI или просто переименованный MVVM?»

- *Сильный ответ:* Полноценная UDF state machine: владельцем состояния является MVICore `ActorReducerFeature`, вход один (`acceptAction`), все мутации централизованы в чистом `Reducer` и применяются последовательно на feature-потоке, one-shot события формализованы как `NEWS`. От MVVM отличают не термины, а рантайм-гарантии: сериализация редукции и воспроизводимость переходов, закреплённая табличными тестами.
- *Типичная ошибка:* «У нас StateFlow и sealed class — значит MVI». Это признаки UDF вообще.
- *Follow-up:* «Что из этого дал бы и дисциплинированный MVVM?» — единый immutable state, SSOT, events-up/state-down. А формализация Actor/Effect/Reducer и сериализация редукции — нет.

Q2. Кто владеет состоянием?

- *Сильный ответ:* `BehaviorSubject` внутри `feature`. `MutableStateFlow` во `ViewModel` — асинхронное зеркало, наполняемое коллектором в `viewModelScope`. Поэтому в тестах сразу после `acceptAction` зеркало может отставать, а при смерти моста `feature` продолжит жить без UI.
- *Типичная ошибка:* «`MutableStateFlow` во `ViewModel` — это и есть `store`».
- *Follow-up:* «Что произойдёт, если мост `state` упадёт?» → см. Q13.

Q3. Зачем в MVICore разделены `Wish` и `Action` — и почему у вас их нет?

- *Сильный ответ:* `Wish` — публичное намерение извне; `Action` — внутренний язык `feature`, его порождают ещё `Bootstrapper` и `PostProcessor`. `ActorReducerFeature` — упрощение с `wishToAction = identity`, поэтому у нас `Wish ≡ Action`, а внутренних действий нет.
- *Типичная ошибка:* «Это синонимы одного и того же».
- *Follow-up:* «Где отсутствие внутренних `Action` начинает мешать?» — стартовая подписка (нет `Bootstrapper`) и цепочки «после X сделай Y» (нет `PostProcessor`).

Q4. Что такое `Bootstrapper`, чем вы его заменяете и какие риски?

- *Сильный ответ:* Источник стартовых `Action` при создании `feature`: `initial load`, подписка на репозиторий. У нас его нет — загрузку иницирует UI через `ScreenOpened`. Два риска: `feature` «мертва» до первого `action`, и `ScreenOpened` прилетает повторно при пересоздании `view` над живой `ViewModel`. Лечится идемпотентным `guard`'ом в `Actor`.
- *Типичная ошибка:* не связать «нет `Bootstrapper`» → «обязательная идемпотентность стартового `action`».
- *Follow-up:* «Почему повторный `ScreenOpened` вообще возможен при живой VM?» — `view-lifecycle` короче `lifecycle ViewModel` (`backstack`, `rotation`).

Q5. Почему `Actor` и `Reducer` «не должны бросать исключения»? Все ли исключения равны?

- *Сильный ответ:* У `feature` нет `error`-канала: `onError` внутренней Rx-подписки уходит в `RxJavaPlugins.onError`; с установленным глобальным `handler`'ом получаем молчаливо мёртвую подписку. Поэтому ожидаемые ошибки материализуются как `EFFECT`. Но `CancellationException` обязан пролетать — иначе ломается `structured cancellation`, — а программные дефекты должны быть `fail-fast` и наблюдаемыми, а не превращаться в вечный спиннер.
- *Типичная ошибка:* «Оборачиваем всё в `runCatching`».
- *Follow-up:* «Что конкретно случится, если `Actor` всё же бросит?» — подписка этого `action` умрёт, ошибка уйдёт в глобальный `handler`, экран «зависнет» без краша.

Q6. Feature однопоточная — значит, запросы выполняются последовательно?

- *Сильный ответ:* Нет. Сериализуется только применение эффектов; на каждый action создаётся отдельная подписка на actor-Observable — plain parallel merge, без `concatMap`/`switchMap`/`exhaustMap`. Отсюда out-of-order и stale results; защита — `requestId`-guard в Reducer, честная отмена — только сменой runtime.
- *Типичная ошибка:* «Однопоточность = FIFO запросов».
- *Follow-up:* «Почему MVICore не сделал concatMap?» — один медленный запрос заблокировал бы весь конвейер, включая независимые действия.

Q7. Что отменяет Actor-операции при закрытии экрана?

- *Сильный ответ:* Цепочка `onCleared` → `feature.release()` → `dispose()` гасит `CompositeDisposable`; у `asObservable` стоит `setCancellable { job.cancel() }`, так что отмена доходит `CancellationException`'ом до suspend-репозитория. Отдельно `viewModelScope` гасит мосты state/news. Это два независимых механизма.
- *Типичная ошибка:* «viewModelScope всё отменит» — он не властен над внутренними подписками feature.
- *Follow-up:* «А отменить один конкретный устаревший запрос?» — точно нельзя: disposable общий; поэтому и нужен `requestId`-guard.

Q8. Что будет, если вызвать `acceptAction` из фонового потока?

- *Сильный ответ:* `featureScheduler = null` → thread affinity к потоку создания; `SameThreadVerifier` бросит исключение внутри Rx-цепочки. С глобальным Rx-handler'ом это выглядит как «экран молча завис», без него — возможен краш через undeliverable. Фикс: `@MainThread` + debug-check, либо явный single-thread scheduler.
- *Типичная ошибка:* «Будет гонка данных / всё сразу упадёт».
- *Follow-up:* «Почему в KDос написано "не упадёт, но работать не будет"?» — это свойство конфигурации проекта (глобальный handler), а не гарантия MVICore.

Q9. Зачем нужен `FeatureScheduler` и почему у вас он фактически недоступен?

- *Сильный ответ:* Он переносит state machine на выделенный однопоточный scheduler — детерминизм без привязки к main. У нас он заблокирован дважды: публичный конструктор всегда передаёт `null`, а `ActorAdapter` жёстко собирает flow на `Main.immediate` — эффекты придут на main и сломают thread-check любого фонового scheduler'a.
- *Типичная ошибка:* «Достаточно пробросить scheduler во внутренний конструктор».
- *Follow-up:* «Что менять, чтобы включить?» — адаптер: `asObservable(scheduler.asCoroutineDispatcher())` + прокинутый scheduler.

Q10. Разница между EFFECT и NEWS? Почему News нельзя через StateFlow?

- *Сильный ответ:* `EFFECT` — вход редьюсера (`Result/Mutation` в канонической терминологии); `NEWS` — одноразовый выход после редукции, не влияющий на `STATE`. `StateFlow` — `conflated + replay` последнего значения: событие либо схлопнется, либо повторится у каждого нового подписчика — обе семантики ломают «ровно один показ».
- *Типичная ошибка:* не проговорить рассинхрон словарей (во многих кодовых базах `Effect = UI-событие`) — и полдиалога спорить о терминах.
- *Follow-up:* «Почему официальный гайд советует события класть в `state`?» — потому что `state+ack` даёт лучшие гарантии; наша `News`-модель — осознанный трейд-офф.

Блок C. News и lifecycle

Q11. Какие гарантии доставки даёт ваш News-механизм?

- *Сильный ответ:* Формальных — никаких: не `at-most-once` (очередь + пересоздание экрана могут показать событие повторно), не `at-least-once` (гонка `check-then-emit`; потеря при отмене во время `dispatchQueue`), следовательно не `exactly-once`. Честная формулировка: `best-effort broadcast` с неограниченным `in-memory` буфером на периоды без подписчика.
- *Типичная ошибка:* «`SharedFlow` гарантирует доставку всем подписчикам».
- *Follow-up:* «Назовите конкретный сценарий потери» — отмена коллектора между проверкой `subscriptionCount` и `emit`.

Q12. Что происходит с News при повороте экрана? При process death?

- *Сильный ответ:* `Rotation`: короткое окно без коллектора закрывается очередью — ради этого она и написана; обратная сторона — при возврате на экран очередь воспроизведёт устаревшие события, включая навигацию. `Process death`: очередь и `state` живут в памяти — всё исчезает, `VM` стартует с `initialState`; критичные события — только `durable` (`state+ack` поверх `SavedStateHandle`, `Room`).
- *Типичная ошибка:* считать, что `ViewModel` переживает `process death`.
- *Follow-up:* «Как проверить руками?» — `adb shell am kill` из `background`; «`Don't keep activities`» недостаточно.

Q13. Чем опасен `catch { Timber.e }` на мостах `feature` → `ViewModel`?

- *Сильный ответ:* `catch` завершает `flow`: первая же ошибка убивает мост навсегда. Получаем «полуживую» `ViewModel`: `feature` жива и принимает `actions`, а `StateFlow` заморожен — пользователь видит вечно висящий экран, ошибка существует только в логах. Фикс: `reportNonFatal` + `rethrow` в `debug` — отказ становится наблюдаемым.
- *Типичная ошибка:* «`catch` защищает от крашей, значит всё хорошо».
- *Follow-up:* «Поймает ли этот `catch` исключение `Actor'a`?» — нет: ошибки `Actor'a` живут во внутренней подписке `feature` и до моста не доходят.

Q14. Как правильно сделать навигацию как одноразовое событие?

- *Сильный ответ:* В нашем стеке News собирает Fragment (он ближе всего к FragmentAttacher) в `repeatOnLifecycle(STARTED)`. Но для навигации надёжнее durable state + acknowledgement: `pendingNavigation(id)` в `STATE`, UI выполняет переход и шлёт `NavigationHandled(id)` — переживает пересоздание и не реплеится устаревшим. Channel-подход одноразов, но только пока consumer один.
- *Типичная ошибка:* `collectAsStateWithLifecycle` для событий.
- *Follow-up:* «Почему STARTED-коллекция усугубляет replay?» — в фоне подписчика нет, события копятся и выстреливают пачкой при возврате.

Блок D. Compose

Q15. `collectAsState` vs `collectAsStateWithLifecycle` — и почему для news ни то ни другое?

- *Сильный ответ:* `WithLifecycle` останавливает коллекцию вне `STARTED` — рекомендованный Android-способ для экранов; `collectAsState` — platform-agnostic и собирает, пока жива композиция. Для событий оба не годятся: «событие как state» либо conflated, либо реплеится новому подписчику; нужен активный одноразовый `collect` внутри `repeatOnLifecycle`.
- *Типичная ошибка:* не знать, что `collectAsStateWithLifecycle` живёт в отдельном артефакте `lifecycle-runtime-compose`.
- *Follow-up:* «Когда разница между ними реально проявится?» — фоновые обновления: с `WithLifecycle` upstream не жжёт работу, пока экран в фоне.

Q16. Что вы делаете со стабильностью STATE для рекомпозиции?

- *Сильный ответ:* `STATE` — immutable data class; коллекции — `ImmutableList` или `@Immutable`-обёртка, потому что `STATE` пересекает границы модулей, где инференс стабильности не работает; колбэки — method reference `viewModel::acceptAction`. Strong skipping (по умолчанию с Kotlin 2.0.20) смягчает цену unstable-параметров, но контракт надёжнее надежды на компилятор.
- *Типичная ошибка:* «List стабильный — это же read-only интерфейс».
- *Follow-up:* «Что strong skipping даёт лямбдам?» — автоматическую мемоизацию даже с unstable capture.

Блок E. Эволюция

Q17. Unchecked cast в `WidgetMviViewModel` — в чём проблема и как чинить?

- *Сильный ответ:* Интерфейс обещает принимать любой `WidgetAction`, реализация умеет только свой подтип; из-за type erasure каст молчит, а `ClassCastException` взрывается позже в Actor/Reducer со стектрейсом не на виновника. Фикс — `generic ViewModel<ACTION : WidgetAction>`; если фреймворк требует негенерик — явный валидирующий адаптер на границе, а не `@Suppress` в базовом классе. Плюс `onCommand` обязан идти через `mapCommandToAction` — иначе второй вход ломает UDF.
- *Типичная ошибка:* «Suppress — нормально, там всегда правильный тип прилетает».
- *Follow-up:* «Почему CCE не в месте вызова acceptAction?» — erasure: каст к generic-типу не проверяется до конкретного использования значения.

Q18. Ваша стратегия эволюции этой архитектуры?

- *Сильный ответ:* Три горизонта. Сначала укрепить контракты, не меняя API: `@MainThread`, `requestId`, bounded/durable news, наблюдаемые отказы мостов, `SavedStateHandle`, generic widget. Затем изолировать Rx в одном модуле и перейти от наследования к композиции (`MviFeatureHost`) — runtime становится заменяемым. Дальше новые экраны — на coroutine-native reducer loop с сохранением типов `Action`/`Effect`/`State` и всех табличных тестов Reducer. Big-bang-переписывание — худший вариант: контракты хорошие, менять нужно runtime.
- *Типичная ошибка:* «Перепишем всё на <модный фреймворк>».
- *Follow-up:* «Что вы сохраняете любой ценой при миграции?» — типы контрактов и тесты Reducer: они и есть спецификация поведения.

Питчи и шпаргалка

Питч на 30 секунд

В проекте UDF/MVI на базе Badoo MVICore. Feature принимает Action; Actor выполняет асинхронную работу и возвращает Flow эффектов; чистый Reducer последовательно применяет их к единому immutable State; NewsPublisher после редукции публикует одноразовые события. Rx-движок скрыт за anti-corruption layer: наружу ViewModel отдаёт `StateFlow` и `Flow<News>`, а в `onCleared` освобождает feature. Compose собирает state через `collectAsStateWithLifecycle`, события обрабатывает Fragment рядом с навигацией.

Питч на 60 секунд — с оценкой (это и отличает Senior)

...Сильные стороны: формализованная state machine, сериализованная редукция, табличные тесты Reducer, контракты не зависят от Rx. Я знаю три слабых места этой реализации и их фиксы. Первое — thread ownership неявный: feature привязана к потоку создания, лечится `@MainThread`-контрактом или явным scheduler'ом. Второе — сериализация редукции не равна сериализации запросов: Actor'ы параллельны, от stale results защищает `requestId`-guard в Reducer. Третье — собственная News-очередь даёт best-effort, а не exactly-once: для снэкбаров достаточно Channel с bounded-буфером, для навигации — durable state с acknowledgement. Стратегия развития: укрепить контракты → изолировать Rx за композицией → новые экраны на coroutine-native UDF, сохраняя типы и тесты.

Шпаргалка за 30 минут до интервью

Поток данных одной строкой: `UI → acceptAction (main) → Feature.accept → Actor(snapshot) → Flow<EFFECT> → asObservable(Main.immediate) → Reducer (последовательно) → BehaviorSubject → asFlow → StateFlow → Compose`; после редукции: `NewsPublisher → NEWS → очередь/SharedFlow → Fragment → FragmentAttacher`.

Шесть слабых мест ↔ фиксы:

Слабое место	Фикс
Thread ownership неявный	<code>@MainThread</code> + debug-check; долгосрочно — явный <code>FeatureScheduler</code> или coroutine loop
Stale results (Actor'ы параллельны)	<code>requestId</code> в STATE/EFFECT + guard в Reducer
News — best-effort	<code>Channel(64, DROP_OLDEST)</code> для снэкбаров; durable state + ack для навигации
«Полуживая» VM (на мостах)	<code>reportNonFatal</code> + rethrow в debug
Process death → <code>initialState</code>	<code>SavedStateHandle</code> recovery input (запись из подписки во VM, не из Reducer)
Widget unchecked cast + <code>onCommand</code>	generic <code>WidgetViewModel<ACTION></code> + <code>mapCommandToAction</code>

Три стратегии (по горизонтам): укрепить контракты, не меняя API → изолировать Rx в одном модуле, композиция `MviFeatureHost` вместо наследования → новые экраны coroutine-native с теми же типами и тестами.

Фразы-маркеры сеньорности:

- «Сериализация редукции $\neq$ сериализация запросов».
 - «Effect — вход редьюсера, News — одноразовый выход».
 - «Доставка News — best-effort, не exactly-once».
 - «CancellationException не глотаем — иначе ломается structured cancellation».
 - «Владелец состояния — feature, StateFlow — асинхронное зеркало».
 - «Wish $\equiv$ Action, потому что ActorReducerFeature».
-

Приоритетный чек-лист внедрения

P0 — до первого инцидента:

1. `@MainThread` + debug-check на `acceptAction`.
2. `requestId`-guard во всех «загрузочных» фичах (эффекты успеха и ошибки).
3. Ограничить News: `Channel(64, DROP_OLDEST)` или `bounded SharedFlow`; навигация — через state + ack.
4. Сделать отказ мостов наблюдаемым: `reportNonFatal` + rethrow в debug.
5. Тест на cancellation через `ActorAdapter` (12.4) — исполняемая гарантия контракта.
6. Убрать unchecked cast в widget-иерархии.

P1 — архитектурная надёжность:

1. `SavedStateHandle` recovery input для ключевых экранов.
2. Явная трёхклассовая модель ошибок Actor'a (Тема 7) вместо «never throw» без оговорок.
3. Публичные потоки — через `asStateFlow()`/`asSharedFlow()`, mutable наружу не отдавать.
4. Widget: `StateFlow` вместо `Flow` в контракте, все команды — через `mapCommandToAction`.
5. Идемпотентность стартовых action — во всех фичах, не только новых.

P2 — постепенная модернизация:

1. Изолировать MVICore/Rx в одном framework-модуле; `MviFeatureHost` (композиция) как новый hosting-контракт.
 2. Новые экраны — coroutine-native reducer loop с теми же типами `Action`/`Effect`/`State`.
 3. Миграция старых фич — только по бизнес-необходимости; табличные тесты Reducer не меняются никогда.
-

Перед интервью: проверьте в своём репозитории

Четыре факта, на которых построена часть выводов конспекта, — освежите их по коду накануне, чтобы отвечать от первоисточника, а не от конспекта:

1. **Точный код** `ActorAdapter` — какой контекст передан в `asObservable` (здесь принято `Dispatchers.Main.immediate` как подтверждённая деталь; если контекст другой — выводы Темы 5.2 скорректируются).
2. **Глобальный** `RxJavaPlugins.setErrorHandler` — установлен ли, и что он делает в `debug` и `release` (от этого зависит вся история «не упадёт, но работать не будет»).
3. **Версия MVICore** в `libs.versions.toml` — 1.x/Rx2 или 2.x/Rx3: влияет на разговор о миграциях.
4. **Нет ли второго коллектора** `news` где-то в проекте (Activity, аналитика) — критично перед переходом на Channel/unicast.

Удачи на собеседовании — с этим материалом вы знаете свою архитектуру глубже, чем большинство её авторов. 🚀